



1.软件开发的本质是为了实现

- A.编写代码
- B.解决问题
- C.使用高级编程语言
- D.设计美观的界面

2.在需求获取过程中，以下哪种技术最适合于揭示用户潜在的、未明确表达的需求？

- A.问卷调查
- B.焦点小组
- C.观察用户的工作流程
- D.文档分析

3.结构化需求分析中，以下哪项是用于描述系统应该做什么，而不是系统如何做的？

- A.数据字典
- B.状态转换图
- C.功能需求
- D.过程规范

4.在软件建模过程中，UML（统一建模语言）是一种常用的工具，以下哪个图是UML中用于描述系统静态结构的主要图？

- A.类图
- B.用例图
- C.序列图
- D.状态图

5.在软件建模的实践中，以下哪种技术通常用于表示系统的动态行为和不同对象之间的交互？

- A.数据流图
- B.状态转换图
- C.顺序图
- D.结构图

6.结构化设计方法中，以下哪项原则指导我们将系统划分为若干个高内聚、低耦合的模块？

- A.模块化原则
- B.信息隐藏原则
- C.逐步求精原则
- D.功能独立性原则

7.结构化设计中，以下哪种图用于表示软件系统的模块及其相互关系？

- A.数据流图（DFD）
- B.状态转换图（STD）
- C.结构图（SC）
- D.类图（UML）

8.在UML的类图中，以下哪个符号表示继承关系？

- A.实线三角

- B.虚线三角
- C.实线箭头
- D.虚线箭头

9.以下关于软件测试的说法，正确的是

- A.软件测试是为了证明软件是正确的
- B.软件测试是为了找出软件的所有错误
- C.软件测试是为了证明软件没有错误
- D.软件测试是为了评估软件的质量

10.在面向对象编程中，关于类和对象的关系，以下描述正确的是

- A.类是对象的物理表示，而对象是类的逻辑描述
- B.对象是类的实例，类是对象的模板
- C.类和对象是同一种概念，只是在不同编程语言中叫法不同
- D.对象包含类的所有属性和方法，而类只包含属性

11.在UML中，以下关于泛化关系的描述正确的是

- A.泛化关系表示两个类之间的继承关系，子类可以继承父类的所有属性和方法
- B.泛化关系表示两个类之间的聚合关系，一个类包含另一个类
- C.泛化关系表示两个类之间的实现关系，一个类实现了另一个类的接口
- D.泛化关系表示两个类之间的关联关系，一个类知道另一个类的存在

12.在UML用例图中，以下关于参与者和用例关系的描述错误的是

- A.参与者可以与多个用例进行交互
- B.一个用例可以被多个参与者触发
- C.参与者只能与用例之间有单向的关系
- D.参与者可以直接与另一个参与者发生关系

13.在UML状态图中，以下关于状态和转换的描述错误的是

- A.状态表示对象在某一时刻所处的条件或情况
- B.转换表示对象从一个状态到另一个状态的迁移
- C.状态之间必须有触发事件才能发生转换
- D.转换可以带有监护条件（guardconditions）

14.在RUP中，以下阶段不是RUP的四个主要阶段之一的是

- A.初始阶段
- B.精化阶段
- C.构建阶段
- D.部署阶段

15.在软件详细设计阶段，常用于描述程序中的控制流程的工具是

- A.数据流图（DFD）
- B.程序流程图（PFD）
- C.类图（UML）
- D.结构图（SC）

16.软件开发就是要“弥补”问题域与运行平台之间的“距离”。在从____向运行平台映射的过程中，为控制其复杂性，需要确定多个抽象层，例如____、设计层、实现层和部署层。P16

02333 软件工程 2025年4月 【13005】

- 17.在软件开发领域，系统模型分为两大类，一类称为____，描述系统是什么：另一类统称为软件模型，描述了实现概念模型的软件解决方案。软件模型可进一步分为设计模型、实现模型和____。 P19
- 18.经过多年研究，已经提出了一系列的软件开发方法。典型的方法包括____、面向数据结构的软件开发方法和____。 P35
- 19.结构化程序设计方法是一种基于结构的编程方法，即采用____、选择结构以及重复结构进行编程，其中每一结构只允许一个____和一个出口。 P63
- 20.在UML中，状态图可用于描述一个对象行为的____和状态____。
- 21.所谓成熟度等级，是指达到预先定义的一组过程域所有目标的一种____等级。在CMMI中，应用于一个组织过程改善的成熟度等级有____个。 P257
- 22.设计模型视角下的体系结构描述，是从设计模型的角度描述那些在体系结构方面有意义的制品。RUP为设计层提供了4个术语，分别是：设计类、____、设计子系统和____。 P147
- 23.软件测试的主要目的是为了发现软件中的____，并帮助改进软件产品的____。
- 24.在瀑布模型中，软件开发的各个阶段按照严格的顺序进行，通常是需求分析、____、编码、测试、部署和____。
- 25.在结构分析中，加工是对____变换单元，即它接受输入数据，对其进行____，并产生输出。
- 26.在进行软件系统/产品的需求工作中，通常面临三大挑战。
- 27.结构化方法给出了建立系统功能模型的基本步骤，通常包括四大步，请简述这四个基本步骤。

28.耦合指不同模块之间相互依赖程度的度量。耦合产生的主要因素有3类，请简述这3类因素。

29.UML提供了一系列的图形化工具，可用于软件开发中所需要的各种模型开发。UML的图形化工具可分为两大类，一类是结构图，另一类是行为图。请给出UML中用于结构描述和行为描述的分别是哪些图？

30.简述什么是增量模型，并指出该模型的优势与不足。

31.简述软件工程的基本定义。

32.以下是计算两个整数的最大公约数（GCD）的代码，请基于语句覆盖、分支覆盖和条件组合覆盖的要求，分别给出一组满足测试覆盖要求的测试用例集，并分别说明覆盖了哪些条件或条件组合。

```
public class GCD {  
    public static int calculateGCD(int a, int b) {  
        if (a == 0)  
            return b;  
        else if (b == 0)  
            return a;  
        else if (a == b)  
            return a;  
        else if (a > b)  
            return calculateGCD(a - b, b);  
        else  
            return calculateGCD(a, b - a);  
    }  
}
```

33.顺序图是UML图的一种，用于显示对象之间交互的顺序。考虑网络打印机打印的场景时，我们可能涉及以下对象：

用户（User）：发起打印请求的个人。打印客户端（PrintClient）：用户使用的应用程序或服务，用于发送打印任务。网络（Network）：连接用户和打印机的通信基础设施。打印服务器（PrintServer）：管理打印任务的服务器。打印机（Printer）：执行打印任务的设备。

可能发生的情况有：

- 用户请求打印文档。
- 打印客户端将打印任务发送到打印服务器。
- 打印服务器检查打印机状态。
- 如果打印机空闲，打印服务器将任务发送到打印机。
- 打印机开始打印。
- 打印完成后，打印机通知打印服务器打印完成。
- 打印服务器可能发送打印完成的确认给用户。

请针对此打印场景，给出打印时消息传递序列说明，并且用一个UML的交互顺序图来描述对象之间的交互。

答案

1.B

解析：可以把软件开发的本质概括为：不同抽象层术语之间的“映射”，以及不同抽象层处理逻辑之间的“映射”。P17

2.C

解析：观察用户的工作流程是揭示用户潜在需求的最佳方式，因为它直接关注用户在实际操作中的行为和问题，而不是依赖用户的自我报告。通过观察，可以发现用户在使用过程中遇到的困难、不便之处以及潜在的改进点，这些往往是用户自己没有意识到的。

3.C

解析：题目问的是“描述系统应该做什么，而不是系统如何做”，即需求分析中的“What”（功能目标）而非“How”（实现细节）。

4.A

解析：类图是可视化地表达系统静态结构模型的工具，通常包含类、接口、关联、泛化和依赖关系等。P101

5.C

解析：顺序图是一种交互图，即由一组对象以及按时序组织的对象之间的关系组成，其中还包含这些对象之间所发送的消息。P113

6.D

解析：改进软件结构，提高模块独立性。针对系统的初始模块结构图，在认真审查分析的基础上，通过模块分解或合并，改进软件结构，力求降低耦合，提升内聚，提高模块独立性。P50

7.C

解析：结构图用于表示软件系统的模块及其相互关系。它展示了系统的层次结构、模块之间的调用关系以及模块之间的数据传递。

8.A

解析：实线三角箭头表示继承关系（也称为泛化关系）。它用于表示一个类（子类）继承了另一个类（父类）的属性和方法。

9.D

解析：软件测试的主要目标是通过各种测试手段来评估软件的质量，发现潜在的缺陷和问题，并确保软件能够满足用户的需求。测试可以帮助开发团队了解软件的性能、可靠性、安全性等方面的表现，从而进行改进和优化。

10.B

解析：类是一组具有相同属性、操作、关系和语义的对象的描述。对象是类的一个实例。P77

11.A

解析：泛化关系（也称为继承关系）用于表示一个类（子类）继承了另一个类（父类）的属性和方法。子类继承父类的所有公有属性和方法，并可以扩展或覆盖这些属性和方法。

12.D

解析：在UML用例图中，参与者（Actor）和用例（UseCase）之间的关系是核心内容之一。参与者只能通过用例与系统交互，而不能直接与其他参与者发生关系。参与者之间的直接关系通常在其他图（如活动图或序列图）中表示。

13.C

解析：虽然大多数状态转换是由触发事件（Event）引起的，但状态转换也可以由时间条件（如超时）或内部活动完成（如完成一个操作）触发。因此，并非所有状态转换都需要显式的触发事件。

14.D

解析：在RUP中，规定了4个开发阶段：初始阶段、精化阶段、构造阶段和移交阶段。P125

15.B

解析：程序流程图。程序流程图又称为程序框图。从20世纪40年代末到70年代中期，程序流程图一直是软件设计的主要工具。因此，它是一种历史最悠久、使用最广泛的软件设计工具。它的主要优点是对控制流程的描绘很直观，便于初学者掌握。P64



16.

答：问题域
需求层

17.

答：概念模型
部署模型

18.

答：结构化方法
面向对象方法

19.

答：顺序结构
入口

20.

答：变化
迁移

21.

答：过程改善
5

22.

答：用况细化
接口

23.

答：错误

质量

24.

答：设计

运行

25.

答：数据流

处理

26.

答：(1) 问题空间的理解；

(2) 人与人之间的通信；

(3) 需求的变化性。P35

27.

答：(1) 需求分析；

(2) 系统分析；

(3) 系统设计；

(4) 系统实施。

28.

答：(1) 耦合产生的主要因素；

(2) 一个模块向另一个模块传递数据；

(3) 一个模块对另一个模块施加控制。P56

29.

答：(1) 结构图：类图、对象图、组件图、部署图、包图。

(2) 行为图：用例图、序列图、协作图、状态图、活动图。

30.

答：增量模型是提出的另一种软件生存周期模型。该模型意指需求可以分组，形成一个个的增量，并可形成一个结构，在这一条件下，可对每一增量实施瀑布式开发。

增量模型的突出优点是：

(1) 第一个可交付版本所需要的成本和时间是较少的，从而可减少开发由增量表示的小系统所承担的风险。

(2) 由于很快发布了第一个版本，因此可以减少用户需求的变更。

(3) 允许增量投资，即在项目开始时可以仅对一个或两个增量投资。

但是，如果增量模型不适合于某些项目或使用有误，则会显示以下主要缺点：

(1) 如果没有对用户的变更要求进行规划，那么产生的初始增量可能会造成后来增量的不稳定。

。

(2) 如果需求不像早期思考的那样稳定和完整，那么一些增量就可能需要重新开发，重新发布。

(3) 由于进度和配置的复杂性，可能会增大管理成本，超出组织的能力。P224

31.

答：软件工程是应用计算机科学理论和技术以及工程管理原则和方法，按预算和进度实现满足用户要求的软件产品的工程，或以此为研究对象的学科。P15

32.

答：

为了满足语句覆盖、分支覆盖和条件组合覆盖的要求，我们需要设计不同的测试用例集。首先，让我们分析代码中的条件和分支：

1. `if (a == 0)` : 当 `a` 为 0 时，返回 `b` 。
2. `else if (b == 0)` : 当 `b` 为 0 时，返回 `a` 。
3. `else if (a == b)` : 当 `a` 和 `b` 相等时，返回 `a` 。
4. `else if (a > b)` : 当 `a` 大于 `b` 时，递归调用 `calculateGCD(a - b, b)` 。
5. `else` : 当 `a` 小于 `b` 时，递归调用 `calculateGCD(a, b - a)` 。

语句覆盖 (Statement Coverage)

语句覆盖要求每个可执行语句至少执行一次。以下测试用例集满足语句覆盖要求：

- 测试用例1: `calculateGCD(0, 5)` // 覆盖 `a == 0` 分支
- 测试用例2: `calculateGCD(5, 0)` // 覆盖 `b == 0` 分支
- 测试用例3: `calculateGCD(5, 5)` // 覆盖 `a == b` 分支
- 测试用例4: `calculateGCD(10, 5)` // 覆盖 `a > b` 分支
- 测试用例5: `calculateGCD(5, 10)` // 覆盖 `else` 分支

分支覆盖 (Branch Coverage)

分支覆盖要求每个分支 (`True` 和 `False`) 至少执行一次。以下测试用例集满足分支覆盖要求：

- 测试用例1: `calculateGCD(0, 5)` // 覆盖 `a == 0` 为 `True`
- 测试用例2: `calculateGCD(5, 0)` // 覆盖 `b == 0` 为 `True`
- 测试用例3: `calculateGCD(5, 5)` // 覆盖 `a == b` 为 `True`
- 测试用例4: `calculateGCD(10, 5)` // 覆盖 `a > b` 为 `True`
- 测试用例5: `calculateGCD(5, 10)` // 覆盖 `a > b` 为 `False`，执行 `else` 分支

条件组合覆盖 (Condition Combination Coverage)

条件组合覆盖要求每个条件的所有可能组合至少执行一次。对于这个代码片段，我们需要考虑以下条件组合：

- 1. `a == 0` 和 `b == 0`
- 2. `a == 0` 和 `b != 0`
- 3. `a != 0` 和 `b == 0`
- 4. `a == b`
- 5. `a != b` 且 `a > b`
- 6. `a != b` 且 `a < b`

以下测试用例集满足条件组合覆盖要求：

- 测试用例1: `calculateGCD(0, 5)` // 覆盖 `a == 0` 和 `b != 0`
- 测试用例2: `calculateGCD(5, 0)` // 覆盖 `a != 0` 和 `b == 0`
- 测试用例3: `calculateGCD(5, 5)` // 覆盖 `a == b`
- 测试用例4: `calculateGCD(10, 5)` // 覆盖 `a != b` 且 `a > b`
- 测试用例5: `calculateGCD(5, 10)` // 覆盖 `a != b` 且 `a < b`

33.
答：

